

XNC: XOR and NOT-Based Lossless Compression for Optimizing Unquantized Embedding Layers in Large Language Models

Junghyeok Lee, Jihoon Jang, and Hyun Kim

Department of Electrical and Information Engineering and Research Center for Electrical and Information Technology
Seoul National University of Science and Technology
Seoul, Korea

ljhy20k@seoultech.ac.kr, jjihoon@seoultech.ac.kr, hyunkim@seoultech.ac.kr

Abstract—Although 4-bit quantized small LLMs have been proposed recently, many studies have retained FP16 precision for embedding layers, as they constitute a relatively small proportion of the overall model in existing LLMs and suffer from severe accuracy degradation when quantized. However, in quantized small LLMs, the embedding layer accounts for a substantial proportion of the total model parameters, necessitating its compression. Since embedding layers are sensitive to approximation, lossless compression is more desirable than lossy compression methods such as quantization. While existing lossless compression methods efficiently compress patterns such as zeros, narrow values, or frequently occurring values, embedding layers typically lack these patterns, making effective compression more challenging. In this paper, we propose XOR and NOT-based lossless compression (XNC), which applies XOR operations between adjacent 16-bit blocks and then performs a NOT operation on the result, effectively truncating the upper and lower bits to compress the embedding layer to 9-bit without any loss. The proposed method leverages XOR and NOT operations, enabling easy hardware implementation, with only four cycles required for compression and three cycles for decompression, ensuring efficient data compression without performance degradation. As a result, the proposed compression technique achieves an average compression ratio of $1.34\times$ for the embedding layers of small LLMs without any loss, effectively reducing the model size of 4-bit quantized LLMs by an average of 9.91%. The code is available at <https://github.com/IDSL-SeoulTech/XNC>.

Index Terms—Lossless compression, embedding layer, small language models, quantization, on-device AI

I. INTRODUCTION

Recently, on-device AI, which processes data directly within the device, has gained significant attention due to its advantages in enhancing data security, reducing latency, and alleviating the heavy computational load on cloud servers [1]. In line with this trend, smartphone vendors are advancing hardware (HW) by increasing DRAM capacity and improving NPU performance [2], [3] and AI companies are actively releasing small large language models (LLMs) (e.g., Llama-3.2 [4], Gemma-2 [5], Qwen-2.5 [6], Phi-3.5 [7], SmoLLM [8]). Nevertheless, due to the HW resource constraints imposed by the limited DRAM and on-chip memory capacity of mobile devices [9], [10], additional compression methods are essential for deploying these small LLMs effectively [11], [12].

Accordingly, various model compression methods have been proposed to reduce the size of LLMs and enable efficient inference [13]–[15]. Among these, quantization is one of the most prominent methods, particularly with recent methods that quantize model weights to 4-bit precision. In existing LLMs, the embedding layer typically accounts for a smaller proportion of the model, and because quantization often results in significant accuracy degradation, it is common practice to maintain FP16 precision for embedding layers in state-of-the-art quantization methods [16]–[19]. However, in small LLMs, the proportion of the embedding layer increases significantly, and when weights are quantized to 4-bit, the embedding layer proportion becomes too substantial to ignore.

To address the aforementioned challenges, we propose a lossless compression method for embedding layers. Existing lossless compression methods, such as base-delta-immediate (BDI) [20] and bit-plane compression (BPC) [21], primarily focus on patterns like zeros, narrow values, and frequently occurring values. However, these approaches have limited effectiveness when applied to embedding layers. Therefore, we introduce two lossless compression methods for embedding layers, as outlined in the following contributions.

- We propose a lossless 4-bit truncated compression (4TC) that truncates bits with predominantly zeros by analyzing data patterns at the data block level. Since 4TC performs compression on a per-block basis, it is well-suited for HW implementation and has the advantage of reducing metadata overhead by using metadata per block.
- Building upon 4TC, we propose an XOR and NOT-based lossless compression (XNC) method, which significantly improves the compression ratio by increasing the occurrence of frequent patterns through the transformation of embedding layers leveraging XOR and NOT operations.
- XNC achieves compression and decompression in 4 and 3 cycles, respectively, enabling efficient data compression without system performance degradation. When applied to the embedding layers of small LLMs, XNC achieved an average compression ratio of $1.34\times$, leading to a 9.91% reduction in the size of 4-bit quantized LLMs.

II. BACKGROUND AND MOTIVATION

A. Embedding Layers in Large Language Models

In LLMs, the embedding layer transforms tokenized inputs into fixed-size, high-dimensional vectors that encapsulate semantic and syntactic relationships. This enables LLMs to understand long contexts and generate appropriate outputs in tasks such as text generation, summarization, and translation. Fig. 1 shows the proportion of embedding layers in the pre-trained weights of several small LLMs. The reason for the significant proportion of the embedding layer in the small LLM is that the model increased the size of the vocabulary by expanding the number of tokens for better performance of the downstream task [4]. This expansion results in a substantial increase in the parameters of the embedding layer, leading to a higher proportion of embedding layers. Particularly, when 4-bit quantization is applied, the embedding layer remains unquantized and retains FP16 precision, causing its proportion to increase relative to the overall model size. To address this issue, recently released small LLMs have adopted the weight tying method [22], where the embedding layer (`embed_token`) and the language modeling head (`lm_head`) share the same parameters, reducing the embedding layer parameters by 50%. Nevertheless, in 4-bit quantized Llama-3.2-1B, Gemma-2-2B, and Qwen-2.5-0.5B models, the embedding layer still accounts for more than half of the total parameters, occupying 50.95%, 52.86%, 59.41%, respectively.

B. Lossless Compression Methods

BDI [20] leverages the numerical similarity of all values in a data block by setting the first value as a base and storing the remaining values as deltas in a low-bit format. Building on base-delta compression, methods like BPC [21] and GBDI [23] have been proposed to enhance compression ratios. BPC exploits the characteristic that most higher-order bits are zeros and introduces a Delta-BitPlane-XOR method, where the most significant bits (MSBs) of base-delta transformed values are transposed into the first row. GBDI, on the other hand, introduces a method that groups data via histogram binning and selects the statistically optimal base, which can be used as a global base across multiple blocks. FPC [24] compresses data by defining frequent patterns such as sign-extension, upper half-zero, and all-zero patterns. However, statistical methods like histogram binning and entropy encoding [25] require prior data profiling, and base-delta lossless compression becomes less effective when the values are not numerically similar.

Table I presents the analysis results of data patterns in the FP16 embedding layers of various small LLM models. The embedding layer weights, which lack zero and narrow patterns, exhibit low compression efficiency when existing lossless compression methods are applied, as the values are not numerically similar. However, our analysis shows that specific bit positions (*i.e.*, 2nd, 14th, 15th, 16th) are zero in over 99% of the cases. This observation was consistent across LLM families with different parameter sizes. Based on these findings, we propose a compression method specialized for the embedding layers of LLMs.

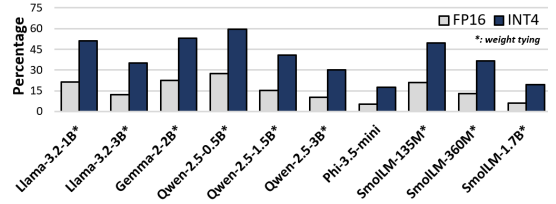


Fig. 1. Percentage of the embedding layer in total model parameters.

TABLE I
DATA PATTERN OF EMBEDDING LAYERS

Model	Llama-3.2-1B	Gemma-2-2B	Qwen-2.5-0.5B	Phi-3.5-mini	SmolLM-135M
All Zero	≈ 0%	≈ 0%	≈ 0%	≈ 0%	0%
Narrow Pattern	0.26%	0.26%	0.26%	0.27%	0.26%
Non Pattern	99.73%	99.73%	99.73%	99.72%	99.74%
2nd,14th,15th,16th bit All Zero	99.83%	99.90%	99.79%	99.75%	99.96%
2nd,14th,15th,16th bit Non-All Zero	0.17%	0.10%	0.21%	0.25%	0.04%

III. PROPOSED XOR AND NOT-BASED LOSSLESS COMPRESSION FOR EMBEDDING LAYERS

A. High-level Overview

Fig. 2 illustrates the overview of the XNC method, which, similar to conventional frame memory compression [26], places a compressor and decompressor between the cache and DRAM to compress data blocks to be stored in DRAM. The compressor employs a *zero checker* module to determine whether a data block is compressible by checking if specific bit positions (*i.e.*, 2nd, 14th, 15th, 16th) of all weights in the block are zero. Based on the *zero checker* result, it determines whether the data block can be compressed to 9-bit or 12-bit, or whether it should be compressed to 12-bit or 16-bit, and then the process is executed in parallel by the *XN encoder*. The *XN encoder* stores the XOR result of the adjacent 16-bit data block and uses the first 16-bit as a base value to remember the original values. The *packers* then combine compressed data of 9-bit, 12-bit, or 16-bit in length with metadata. Finally, the *splitNmerger* module splits the compressed data block into 16-byte, 32-byte, and 48-byte segments, then reassembles them into a 64-byte data block for transfer 64-byte data block to DRAM. The decompressor uses metadata associated with the 32 compressed values to reorder compressed data of varying lengths into a 16-bit format. It then reverses the compression process, restoring the original data and transferring the 64B data block back to the cache.

B. Proposed XNC Method

As mentioned in Section II-B, our data pattern analysis revealed that specific bit positions (*i.e.*, 2nd, 14th, 15th, 16th) in the embedding layer of LLM are zero more than 99% of the time. Based on this observation, we propose the 4TC method, which effectively compresses the embedding layer by truncating these frequently zeroed bits. The 4TC performs a zero check for each data block to identify whether all four specific bits are zero and, if so, compresses the FP16 value into a 12-bit format. To ensure lossless compression, we

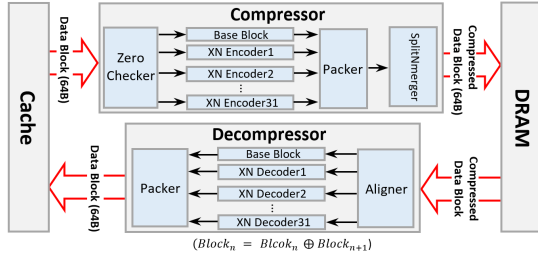


Fig. 2. Overview of the proposed XNC.

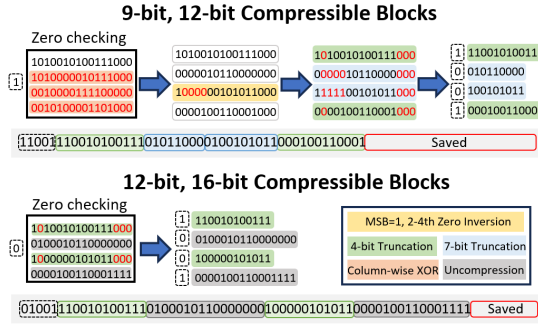


Fig. 3. Examples of XNC data transformation and compression process.

add 1-bit metadata per block to indicate whether truncation has occurred. Given that more than 99% of the specific bit positions in the embedding layer weights are zero, the metadata overhead is minimal.

Additionally, to achieve a higher compression ratio, we propose XNC as an extension of 4TC. Fig. 3 shows an example of compressing an 8-byte data block using XNC. In detail, it illustrates how the FP16 weights of the embedding layer are compressed into a data block. This is accomplished by applying XOR and NOT operations to transform the upper and lower bits into more frequent values, followed by truncation to achieve efficient compression. First, a zero-checking process is performed on each 16-bit data block to determine whether the block can be compressed to 9-bit or 12-bit, or whether it is compressible only to 12-bit or 16-bit. A 9-bit/12-bit compressible block indicates that all data within the block can be compressed to either 9-bit or 12-bit, and the same applies to 12-bit/16-bit compressible blocks. The zero-checked data block is then subjected to a column-wise XOR operation between adjacent 16-bit weights. This XOR operation transforms identical values into zeros, increasing compression efficiency. The first 16-bit is stored without XOR to retain its original value for decoding. After XOR, the data is transformed by applying a NOT operation to the 2nd to 4th bits when the MSB is one and the 2nd to 4th bits are all zeros, ensuring the upper 5 bits form a continuous pattern. The transformed value forms continuous patterns in the upper 5 bits and lower 3 bits, allowing for 9-bit compression by truncating the 4 bits below the MSB to distinguish between cases where the upper 5 bits are all zeros or all ones and truncating the lower 3 bits. If 9-bit compression is not feasible, the block is compressed to 12-bit

TABLE II
XNC ENCODING TABLE

Encoding Type	Metadata	Code
9-bit/12-bit Compressible block	Per block	1'b1 + Data block
12-bit/16-bit Compressible block		1'b0 + Data block
9-bit Compression	Per weight	1'b0 + 9bit
12-bit Compression		1'b1 + 12bit
Uncompressed		1'b0 + 16bit

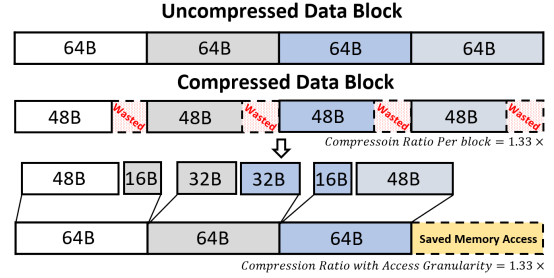


Fig. 4. Example of memory access savings through compression.

using the 4TC method by truncation-specific bits. In the case of 12-bit/16-bit compressible blocks, the data is compressed to 12-bit using 4TC. If the block contains non-zero bits in the specified positions, it is stored in its original 16-bit.

In XNC, metadata is added to differentiate between compressed blocks and compression types. As shown in Table II, each 64-byte data block includes 1 bit of metadata to distinguish between 9-bit/12-bit and 12-bit/16-bit compressible blocks. Additionally, 1 bit of metadata per compressed weight is appended to indicate the encoding type. Please note that both 9-bit and uncompressed weights share the same 1'b0 metadata, but since the block metadata has already categorized them, this does not present any issue. After compression, the 1-bit metadata for the compressible block type and 32 bits of metadata for the encoding types of the 32 compressed weights are stored as a header at the start of the compressed block.

C. Compression Ratio Enhancement with Access Granularity

Fig. 4 shows how compressed data blocks are reorganized to reduce memory accesses, considering DRAM's 64-byte access granularity. Normally, reading 256 bytes requires four accesses. If unused space from compression is not optimized during data transfers, it leads to inefficiency and negates the benefits of compression. To address this, XNC reconstructs 48-byte compressed blocks into 64-byte blocks using the *splitNmerger* module. The first block stays the same, while the remaining blocks are split into 16-byte and 32-byte segments and merged into 64-byte segments. This reduces memory accesses from four to three or fewer, depending on the compression ratio, improving transfer efficiency.

IV. EXPERIMENTAL RESULTS

A. Experimental Environments

We used recently released open-source small LLMs with fewer than 4 billion parameters. AWQ [18] was applied for

4-bit quantization to the pre-trained weights, excluding the embedding and normalization layers. Compression methods used for comparison were adjusted to 16-bit precision and a 64-byte access granularity. Since the metadata is stored in DRAM along with the compressed data, it is also included in the compression ratio.

B. Compression and Decompression Cycles

To evaluate the throughput of the compressor and decompressor, we implemented both the compressor and decompressor in Verilog and measured the cycle count for each operation. For the compressor, the *zero checker* identifies zero bits in a 64-byte data block in one cycle, and the *XN encoder* applies XOR and NOT operations in another cycle. The *packer* then takes two cycles to pack the variable-length compressed data and pass it through the *splitNmerger*, which reorganizes the data into 64-byte blocks. For the decompressor, one cycle is needed to rearrange the 32 compressed data blocks into the 16-bit format, followed by one cycle each for the *XN decoder* and *packer*. As a result, the total compression and decompression latencies for XNC are 4 and 3 cycles, respectively. Due to the application of the method considering access granularity, the first data block incurs a slight additional cycle delay while waiting for the next data block. Table III presents the compression and decompression cycles for various lossless compression methods. XNC demonstrates fewer cycles for both compression and decompression compared to existing methods except for BDI. Notably, the reduced decompression latency makes it suitable for systems prioritizing high throughput and computational efficiency, as decompression latency is a key factor in determining overall system performance.

C. Compression Ratio and Compressed Small LLM Size

Fig. 5 shows the compression ratio results of applying various lossless compression methods to the FP16 embedding layer. The proposed XNC achieved the highest compression ratio, averaging $1.34\times$. Existing lossless compression methods are less effective for embedding layers because they contain few zeros and lack repetitive value patterns. For instance, in the SmolLM-135M embedding layer, the most frequent value accounts for only 0.17%, making statistical methods like GBDI ineffective. In fact, BDI, FPC, and GBDI increased the embedding layer size due to metadata overhead. BPC, on the other hand, demonstrated some effectiveness by compressing consecutive rows of all-zero values, especially in the lower 3 bits, which often contain zeros. Among existing methods, BPC was the only one that achieved notable compression.

Table IV shows a comparison of the sizes of 4-bit quantized small LLMs and vision-language models (VLMs) before and

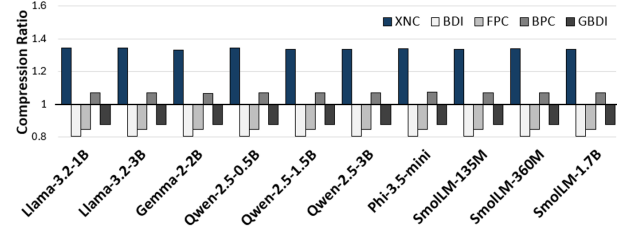


Fig. 5. Compression ratio comparison of each lossless compression method.

TABLE IV
COMPRESSION % OF SMALL LLMs AND VLMs BY THE PROPOSED XNC

Model	4-bit Quantized Model Size (MB)	Model Size after XNC (MB)	Compression Percentage(%)
Llama-3.2-1B	1031	897	13.0
Llama-3.2-3B	2253	2052	8.9
Gemma-2-2B	2232	1937	13.2
Qwen-2.5-0.5B	458	388	15.3
Qwen-2.5-1.5B	1147	1030	10.2
Qwen-2.5-3B	2064	1907	7.6
Phi-3.5-mini	2277	2177	4.4
SmolLM-135M	114	100	12.3
SmolLM-360M	257	233	9.3
SmolLM-1.7B	1038	987	4.9
SmolVLM-256M	299	271	9.4
Qwen-2.5-VL-3B	3226	3069	4.9

after applying XNC. By lossless compressing the FP16 embedding layer to 9-bit or 12-bit using XNC, the total number of parameters in the 4-bit quantized LLMs was reduced by an average of 9.91%, with the Qwen-2.5-0.5B model achieving a size reduction of up to 15.3%. Additionally, in small VLMs [27], [28], where the embedding layer constitutes a substantial portion of the overall parameters, XNC also achieved a high compression ratio, thereby demonstrating its scalability in multimodal applications.

V. CONCLUSIONS

This paper proposes a lossless compression solution for the embedding layer in LLMs. Along with 4TC, which truncates bit positions that are mostly zeros, the XNC enhances the compression ratio by performing XOR operations between adjacent 16-bit blocks and applying a NOT operation to effectively truncate frequently occurring upper and lower bits. XNC achieves an average compression ratio of $1.34\times$ in the embedding layers of recent small LLMs and further improves data transfer efficiency by minimizing memory accesses.

ACKNOWLEDGMENT

This research was partly supported by National R&D Program through the National Research Foundation of Korea(NRF) funded by Ministry of Science and ICT(2020M3H2A1078119) and K-CHIPS(Korea Collaborative & High-tech Initiative for Prospective Semiconductor Research)(2410000620, RS-2024-00405946, 24052-15TC) funded by the Ministry of Trade, Industry & Energy (MOTIE, Korea).

TABLE III
REQUIRED CYCLE FOR THE COMPRESSOR AND DECOMPRESSOR

Compression Method	BDI	FPC	BPC	GBDI	XNC
Compressor Latency (Cycle)	1	9	9	12	4
Decompressor Latency (Cycle)	1	9	9	9	3

REFERENCES

- [1] D. Chun, S. Lee, and H. Kim, "Usd: Uncertainty-based one-phase learning to enhance pseudo-label reliability for semi-supervised object detection," *IEEE Transactions on Multimedia*, vol. 26, pp. 6336–6347, 2024.
- [2] Apple, "iphone 16 pro max," 2024, [Online]. Available: <https://www.apple.com/iphone-16-pro/specs>.
- [3] Samsung, "Galaxy s24 ultra," 2024, [Online]. Available: <https://www.samsung.com/uk/smartphones/galaxy-s24-ultra/specs/>.
- [4] A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Yang, A. Fan *et al.*, "The llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024.
- [5] G. Team, M. Riviere, S. Pathak, P. G. Sessa, C. Hardin, S. Bhupatiraju, L. Hussenot, T. Mesnard, B. Shahriari, A. Ramé *et al.*, "Gemma 2: Improving open language models at a practical size," *arXiv preprint arXiv:2408.00118*, 2024.
- [6] A. Yang, B. Yang, B. Hui, B. Zheng, B. Yu, C. Zhou, C. Li, C. Li, D. Liu, F. Huang *et al.*, "Qwen2 technical report," *arXiv preprint arXiv:2407.10671*, 2024.
- [7] M. Abdin, S. A. Jacobs, A. A. Awan, J. Aneja, A. Awadallah, H. Awadalla, N. Bach, A. Bahree, A. Bakhtiari, H. Behl *et al.*, "Phi-3 technical report: A highly capable language model locally on your phone," *arXiv preprint arXiv:2404.14219*, 2024.
- [8] L. B. Allal, A. Lozhkov, E. Bakouch, L. von Werra, and T. Wolf, "Smollm-blazingly fast and remarkably powerful," *Hugging Face Blog*, 2024, [Online]. Available: <https://huggingface.co/blog/smollm>.
- [9] J. Lee, J. Jang, J. Lee, D. Chun, and H. Kim, "Cnn-based mask-pose fusion for detecting specific persons on heterogeneous embedded systems," *IEEE Access*, vol. 9, pp. 120 358–120 366, 2021.
- [10] H. Hong, D. Choi, N. Kim, H. Lee, B. Kang, H. Kang, and H. Kim, "Survey of convolutional neural network accelerators on field-programmable gate array platforms: architectures and optimization techniques," *Journal of Real-Time Image Processing*, vol. 21, no. 3, p. 64, 2024.
- [11] N. J. Kim, J. Lee, and H. Kim, "Hyq: Hardware-friendly post-training quantization for cnn-transformer hybrid networks," in *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence, IJCAI-24*. International Joint Conferences on Artificial Intelligence Organization, 8 2024, pp. 4291–4299.
- [12] J. Lee and H. Kim, "Dct-vit: High-frequency pruned vision transformer with discrete cosine transform," *IEEE Access*, vol. 12, pp. 80 386–80 396, 2024.
- [13] Z. Liu, M. Sun, T. Zhou, G. Huang, and T. Darrell, "Rethinking the value of network pruning," *arXiv preprint arXiv:1810.05270*, 2018.
- [14] E. Frantar, S. Ashkboos, T. Hoeffler, and D. Alistarh, "Gptq: Accurate post-training quantization for generative pre-trained transformers," *arXiv preprint arXiv:2210.17323*, 2022.
- [15] G. Hinton, "Distilling the knowledge in a neural network," *arXiv preprint arXiv:1503.02531*, 2015.
- [16] Y. Lin, H. Tang, S. Yang, Z. Zhang, G. Xiao, C. Gan, and S. Han, "Qserve: W4a8kv4 quantization and system co-design for efficient llm serving," *arXiv preprint arXiv:2405.04532*, 2024.
- [17] S. Ashkboos, A. Mohtashami, M. L. Croci, B. Li, M. Jaggi, D. Alistarh, T. Hoeffler, and J. Hensman, "Quarot: Outlier-free 4-bit inference in rotated llms," *arXiv preprint arXiv:2404.00456*, 2024.
- [18] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han, "Awq: Activation-aware weight quantization for on-device llm compression and acceleration," *Proceedings of Machine Learning and Systems*, vol. 6, pp. 87–100, 2024.
- [19] Z. Liu, C. Zhao, I. Fedorov, B. Soran, D. Choudhary, R. Krishnamoorthi, V. Chandra, Y. Tian, and T. Blankevoort, "Spinquant-llm quantization with learned rotations," *arXiv preprint arXiv:2405.16406*, 2024.
- [20] G. Pekhimenko, V. Seshadri, O. Mutlu, P. B. Gibbons, M. A. Kozuch, and T. C. Mowry, "Base-delta-immediate compression: Practical data compression for on-chip caches," in *Proceedings of the 21st international conference on Parallel architectures and compilation techniques*, 2012, pp. 377–388.
- [21] J. Kim, M. Sullivan, E. Choukse, and M. Erez, "Bit-plane compression: Transforming data for better compression in many-core architectures," *ACM SIGARCH Computer Architecture News*, vol. 44, no. 3, pp. 329–340, 2016.
- [22] O. Press and L. Wolf, "Using the output embedding to improve language models," *arXiv preprint arXiv:1608.05859*, 2016.
- [23] A. Angerd, A. Arelakis, V. Spiliopoulos, E. Sintorn, and P. Stenström, "Gbdi: Going beyond base-delta-immediate compression with global bases," in *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2022, pp. 1115–1127.
- [24] A. R. Alameldeen and D. A. Wood, "Adaptive cache compression for high-performance processors," *ACM SIGARCH Computer Architecture News*, vol. 32, no. 2, p. 212, 2004.
- [25] S. Lal, J. Lucas, and B. Juurlink, "E² 2mc: Entropy encoding based memory compression for gpus," in *2017 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 2017, pp. 1119–1128.
- [26] H. Kim and H.-J. Lee, "A low-power surveillance video coding system with early background subtraction and adaptive frame memory compression," *IEEE Transactions on Consumer Electronics*, vol. 63, no. 4, pp. 359–367, 2017.
- [27] M. Andres, N. Merve, F. Miquel, B. Elie, and C. Pedro, "Smolvlm-small yet mighty vision language model," *Hugging Face Blog*, 2024, [Online]. Available: <https://huggingface.co/blog/smolvlm>.
- [28] Q. Team, "Qwen2.5-vl," January 2025. [Online]. Available: <https://qwenlm.github.io/blog/qwen2.5-vl/>